

Microservices, Kubernetes & DevOps

Skalierbare digitale Prozessarchitektur bauen –
von Servicegrenzen zu einem belastbaren Operating Model.

Lehrskript – Tag 10 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Am Ende dieses Tages verstehen Sie, warum eine skalierbare digitale Prozessarchitektur nicht aus Werkzeugen, sondern aus *Entscheidungen* besteht. Sie können Servicegrenzen entlang fachlicher Capabilities ziehen, ein Kubernetes-Betriebskonzept skizzieren, eine Teststrategie nach TDD-Prinzipien aufstellen und ein Operating Model entwerfen, das Ownership und Governance leichtgewichtig, aber verbindlich verankert. Drei Stufen – Wissen, Anwendung, Management – bauen aufeinander auf.

L1 – Wissen (Reproduktion und Einordnung)

- Microservices-Grundlogik kennen: unabhängige Deployments, Servicegrenzen entlang Capabilities, API/Event-Integration, Idempotenz.
- Container und Kubernetes konzeptionell einordnen: Deploy, Skalierung, Self-Healing, Rollback, Service-Discovery, Konfiguration/Secrets.
- TDD-Zyklus (Red-Green-Refactor) und Testpyramide (Unit, Integration, End-to-End) als Qualitätsarchitektur verstehen.
- DevOps-Praktiken und Operating-Model-Bausteine (Produktteams, Plattformteam, Security, Architektur-Governance) einordnen.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Einen Monolithen entlang Capabilities in fünf bis acht Service-Kandidaten zerlegen und Steckbriefe formulieren.
- Einen Release-Flow mit zwei Quality Gates definieren und eine Testpyramide pro Service dimensionieren.
- Ein Kubernetes-Betriebskonzept (Rollout/Rollback, Autoscaling, Monitoring/Alerting) auf einen Fallprozess anwenden.
- Ein Operating Model entwerfen, das Produktteams, Plattformteam und Security in einer RACI darstellt.

L3 – Management (Entscheidung und Verantwortung)

- Top-10-Entscheidungen für eine skalierbare Plattform priorisieren (Servicegrenzen, Datenhoheit, SLOs, Standards).
- Zielkonflikte Speed vs. Safety, Autonomie vs. Standardisierung, Innovation vs. Audit transparent entscheiden.
- Reliability- und Risiko-Architektur (SLOs, Error Budgets, Change Failure Rate, MTTR) als Steuerungsinstrument nutzen.
- Entscheidungen unter Unsicherheit mit Frühwarnsignalen absichern statt sie zu vermeiden.

Roter Faden

Microservices sind kein *Technologiethema*, sondern eine *Organisationsentscheidung*. Kubernetes löst die Skalierung – aber nur, wenn die Organisation Skalierung verantworten kann. Wer die Plattform baut, ohne das Operating Model zu klären, baut ein leistungsfähigeres Chaos. (vgl. Newman, 2021,

Kap. 1; Skelton & Pais, 2019)

1. Einstieg: Warum klassische Architektur an Grenzen stößt

Die meisten gewachsenen Unternehmen kennen die gleiche Situation: ein monolithisches Kernsystem, das vor zehn bis fünfzehn Jahren begonnen wurde, mittlerweile alle Geschäftsprozesse trägt, und in jedem Release alle Bereiche gleichzeitig betrifft. Die Release-Zyklen sind lang (sechs bis acht Wochen sind keine Seltenheit), die Tests aufwendig, die Schmerzgrenze der Beteiligten hoch. Jedes neue Feature trägt das Risiko, etwas anderes zu beschädigen – weshalb Vorsicht zur Tugend wird und Innovation zur Ausnahme. (vgl. Newman, 2021, Kap. 1)

In dieser Situation klingt die Antwort einfach: *Microservices*. Wir zerlegen den Monolithen in kleine, eigenständige Dienste, deployen jeden für sich, skalieren punktgenau und befreien die Teams. Die Realität ist nüchterner. Ein schlecht geschnittener Microservice ist kein Fortschritt, sondern eine teurere Form desselben Problems – nur in mehreren Repositories und mit zusätzlichen Schnittstellenkosten.

1.1 Die drei Schmerzpunkte des Monolithen

1. **Kopplung im Code.** Eine Änderung in einem Modul zwingt zur Vollregression. Tests sind teuer, Deployments selten, Eingriffsrisiko hoch.
2. **Kopplung im Datenmodell.** Ein zentrales Schema, das alle Bereiche teilen. Schemaänderungen erfordern Verhandlungen mit allen Anrainern.
3. **Kopplung in der Organisation.** Ein Release bedeutet ein Synchronisationsfenster für alle Teams – mit Steuerungsaufwand, der mit der Größe quadratisch wächst.

1.2 Warum Microservices keine Silberkugel sind

Microservices versprechen Entkopplung – liefern sie aber nur dann, wenn die Servicegrenzen *entlang fachlicher Capabilities* verlaufen, nicht entlang technischer Schichten oder Datenbanktabellen. Ein typischer Anti-Schnitt: “Datenbank-Service”, “UI-Service”, “Logik-Service”. Drei Services, dieselbe Kopplung – nur jetzt über Netzwerkgrenzen, mit Latenz und neuen Fehlerklassen. (Newman, 2021, Kap. 3; Evans, 2003)

Ein guter Servicegrenzen-Schnitt hingegen folgt der Frage: *Was ist eine eigenständige Geschäftsfähigkeit?* “Order”, “Inventory”, “Shipping”, “Billing” sind Capabilities – sie haben eigene Daten, eigene Entscheidungen, eigene Owner. Wenn eine Capability sich ändert, ändert sich *ein* Service, nicht das halbe Unternehmen.

1.3 Was “skalierbar” wirklich bedeutet

Skalierbarkeit hat drei Dimensionen, die oft verwechselt werden:

- **Last-Skalierung.** Der Service verträgt zehntausend statt hundert Requests pro Sekunde. Eher ein Plattformthema (Kubernetes, Autoscaling).
- **Team-Skalierung.** Mehr Teams können parallel an unterschiedlichen Services arbeiten, ohne sich gegenseitig zu blockieren. Eher ein Architekturthema (Servicegrenzen).
- **Entscheidungs-Skalierung.** Die Organisation kann hundert Entscheidungen

pro Woche treffen, ohne im Architekturboard zu ersticken. Eher ein Governance-Thema (Leitplanken, Autonomie).

Eine Plattform ist erst dann skalierbar, wenn alle drei Dimensionen funktionieren. Wer nur Last skaliert, baut leistungsfähige Engpässe. Wer nur Teams skaliert, produziert Architekturchaos. Wer nur Entscheidungen skaliert, riskiert Inkonsistenz.

Eselsbrücke: L-T-E

L-T-E – die drei Skalierungsdimensionen einer Prozessplattform:

Last – mehr Volumen ohne Performance-Bruch?

Teams – mehr parallele Arbeit ohne Blockaden?

Entscheidungen – mehr lokale Autonomie ohne Inkonsistenz?

Wer nur eine Dimension skaliert, baut Engpässe an den anderen.

2. Microservices: Servicegrenzen als Architekturentscheidung

Die zentrale Frage bei Microservices ist nicht “wie klein” – sondern “wie geschnitten”. Servicegröße ist eine Folge, nicht das Ziel. Das Ziel ist Entkopplung entlang sinnvoller Grenzen.

2.1 Capabilities statt Schichten

Eine **Geschäftsfähigkeit** (Business Capability) ist eine eigenständige Leistung, die das Unternehmen erbringt – etwa “Bestellung annehmen”, “Bonität prüfen”, “Versand auslösen”. Sie ist sprachlich klar abgrenzbar, hat einen typischen Input und Output, und sie hat einen verantwortlichen Owner. (Newman, 2021, Kap. 3)

Ein **Bounded Context** im Sinne von Eric Evans ist die sprachlich abgegrenzte Zone, in der ein bestimmtes Modell konsistent gilt. “Kunde” bedeutet im Vertrieb etwas anderes als im Mahnwesen – und das ist kein Problem, sondern eine notwendige Klärung. Die Servicegrenze verläuft an der Grenze des Bounded Context, nicht innerhalb. (Evans, 2003, Kap. 14)

2.2 Die Servicekarte: ein Steckbrief je Service

Für jeden Service-Kandidaten lohnt sich vor dem ersten Codezeilen ein Steckbrief mit fünf Feldern:

- **Zweck.** Eine Capability in einem Satz. Wenn der Satz drei Konjunktionen enthält, ist es vermutlich kein Service, sondern eine Sammlung.
- **Input.** Welche Daten kommen herein? Welche Trigger? Welche Verträge mit anderen Services?
- **Output.** Welche Daten gehen heraus? Welche Ereignisse erzeugt der Service?
- **Owner.** Wer verantwortet diesen Service in Run und Build? Eine Person, kein Team-Mailverteiler.
- **Kritischster Fehlerfall.** Was passiert, wenn der Service ausfällt? Welcher Geschäftsschaden droht? Welche Eskalation?

2.3 Integration: API und Event

Microservices kommunizieren typischerweise über zwei Muster: **synchrone APIs** (oft HTTP/REST oder gRPC) und **asynchrone Events** (Message Bus, Event Stream). Beide haben Stärken und Schwächen: (Richardson, 2018, Kap. 3)

- **APIs** sind einfach zu verstehen, gut testbar und liefern sofortige Antworten. Schwäche: ein Aufruf-Stack koppelt Services in Verfügbarkeit und Latenz. Wenn drei Services synchron aufgerufen werden, ist die Gesamtverfügbarkeit das Produkt der einzelnen.
- **Events** entkoppeln stärker – Sender und Empfänger müssen nicht gleichzeitig verfügbar sein. Schwäche: höhere Komplexität, Reihenfolgeprobleme, Duplikate möglich – weshalb **Idempotenz** (ein Aufruf mehrfach ausführbar ohne Schaden) Pflicht wird.

Idempotenz als Pflichtdisziplin

In verteilten Systemen werden Nachrichten verloren, dupliziert oder neu zugestellt. Jeder Service muss damit umgehen können – entweder fachlich (eine Bestellung mit gleicher ID wird nicht doppelt angelegt) oder technisch (Deduplizierung über Idempotency-Keys). Ohne Idempotenz wird jede Skalierung früher oder später zu einem Datenintegritätsproblem. (Richardson, 2018, Kap. 3)

2.4 Versionierung von Schnittstellen

Ein Service-Vertrag ist eine Verbindlichkeit. Wer ihn bricht, bricht die Verträge aller Konsumenten – mit unvorhersehbaren Effekten. Drei Disziplinen:

1. **Semantische Versionierung.** Major-Versionen für Breaking Changes, Minor für additive Erweiterungen, Patch für interne Fixes. Konsumenten wissen, was sie erwartet.
2. **Tolerant Reader.** Konsumenten lesen nur die Felder, die sie brauchen, und ignorieren unbekannte Felder. So bricht ein zusätzliches Feld niemanden.
3. **Deprecation-Pfade.** Eine alte Version wird nicht abgeschaltet, sondern markiert. Sechs Monate Übergangszeit sind eher die Untergrenze, nicht die Obergrenze.

2.5 Risiken: was Microservices kosten

Wer Microservices einführt, übernimmt eine Liste von Folgekosten, die in der Tool-Werbung selten ehrlich genannt werden:

- **Verteilte Fehler.** Ein Service-Ausfall propagiert sich durch das Netz. Ohne Resilienz-Patterns (Timeouts, Circuit Breaker, Retries mit Backoff) wird ein lokales Problem zum globalen Ausfall.
- **Observability-Pflicht.** Logs, Metriken und Traces müssen über Servicegrenzen hinweg korreliert werden. Ohne Standard wird Debugging zur Detektivarbeit.
- **Daten-Konsistenz.** Statt einer ACID-Transaktion gibt es **eventuelle Konsistenz** – Daten sind nicht überall sofort gleich. Das ist beherrschbar, aber nicht

selbstverständlich.

- **Organisationskosten.** Mehr Services bedeuten mehr Verträge, mehr Schnittstellen, mehr Koordination. Wer 50 Services für 5 Teams baut, hat ein Problem.

3. Container und Kubernetes: Orchestrierung als Plattform

Container sind nicht “Mini-VMs”. Sie sind ein *Packaging-Format*: eine Anwendung plus all ihre Abhängigkeiten in einem reproduzierbaren Artefakt. Was auf dem Laptop läuft, läuft im Cluster – weil der Container die Umgebung mitbringt. Das löst eine alte Klage (“Bei mir läuft’s”) und macht Deployment zur wiederholbaren Prozedur. (Burns et al., 2022, Kap. 1)

3.1 Kubernetes als Orchestrierung

Kubernetes ist die heutige Standard-Antwort auf die Frage: *Wie betreibt man hunderte Container, die auf Dutzenden Maschinen laufen, ohne den Verstand zu verlieren?* Kubernetes übernimmt die Aufgaben, die früher manuell oder mit Skripten erledigt wurden:

- **Deployment.** Welche Container laufen in welcher Version, auf welcher Maschine?
- **Skalierung.** Bei Last steigt die Anzahl der Instanzen, bei Ruhe sinkt sie – auf Regeln basierend.
- **Self-Healing.** Stirbt ein Container, wird er neu gestartet. Stirbt eine Maschine, werden ihre Container woanders neu gestartet.
- **Rollback.** Eine fehlerhafte Version wird in Minuten zurückgenommen.
- **Service-Discovery.** Services finden einander über logische Namen, nicht über IP-Adressen.

3.2 Konfiguration und Secrets

Eine Anwendung sollte ihre Konfiguration nicht im Code tragen, sondern als externe Eingabe bekommen. Kubernetes unterscheidet:

- **ConfigMaps** – nicht-sensitive Konfiguration (Hostnames, Feature-Flags, Sprache).
- **Secrets** – sensitive Werte (Passwörter, API-Keys, Zertifikate). In der Praxis nie unverschlüsselt im Cluster, sondern aus einem **Vault** (HashiCorp Vault, AWS Secrets Manager o. ä.) bezogen.

3.3 Ressourcen-Limits und Quality of Service

Jeder Container kann mit **Requests** (was er garantiert bekommt) und **Limits** (was er maximal bekommen darf) konfiguriert werden. Wer das überspringt, riskiert zwei Klassen von Ausfällen:

- Ein Container greift sich die ganze Maschine, andere verhungern. (Kein Limit.)
- Eine Maschine ist überbucht, alle Container leiden bei Lastspitzen. (Kein Request.)

3.4 Rollout-Strategien: Blue/Green, Canary, Rolling

Wer wöchentlich oder gar mehrmals täglich produziert, braucht Rollout-Strategien, die das Risiko schon im Deployment begrenzen:

- **Rolling Update.** Die Instanzen werden nach und nach ersetzt. Einfach, aber während des Rollouts läuft Old und New gleichzeitig – das muss der Service vertragen.
- **Blue/Green.** Zwei vollständige Umgebungen, Umschalten per Routing. Höhere Infrastruktur-Kosten, aber sofortiges Rollback per Switch.
- **Canary.** Eine kleine Prozentzahl der Nutzer bekommt die neue Version, wird beobachtet, bei grünem Licht wird ausgerollt. Differenziertes Risiko-Management.

Eselsbrücke: B-C-R

Drei Rollout-Strategien für skalierende Plattformen:

Blue/Green – “zweite Umgebung daneben, dann umschalten”

Canary – “erst 5 %, dann 25 %, dann alle”

Rolling – “alte und neue Version parallel, schrittweise tauschen”

Kein Standard ist überall richtig. Risiko-Klasse + Rollback-Anforderung entscheiden.

3.5 Observability als Pflichtprogramm

Eine verteilte Plattform ohne Observability ist ein Blindflug. Drei Säulen: (Burns et al., 2022, Kap. 16; Kim et al., 2016, Teil II)

- **Logs.** Strukturierte, korrelierte Events. Zentral abgelegt, durchsuchbar, mit Korrelations-IDs über Servicegrenzen.
- **Metriken.** Zeitliche Aggregationen: Antwortzeit, Fehlerrate, Auslastung, Queue-Länge. Anhand SLIs (Service Level Indicators) gemessen, gegen SLOs verglichen.
- **Traces.** Der Pfad einer Anfrage durch mehrere Services. Ohne Tracing ist verteiltes Debugging Detektivarbeit.

4. TDD und Teststrategie: Qualität als Architektur

Tests sind nicht etwas, das man “nachher” macht. Sie sind Teil des Architekturwerks. Eine skalierbare Plattform braucht ein Test-System, das sich selbst skaliert – sprich: Tests, die schnell laufen, wenig kosten und trotzdem belastbar sind.

4.1 TDD: Red-Green-Refactor

Test-Driven Development (TDD) folgt einem strikten Zyklus: *Red* (schreibe einen Test, der scheitert), *Green* (schreibe den minimalen Code, damit der Test besteht), *Refactor* (verbessere die Struktur, ohne Verhalten zu ändern). (Beck, 2002)

Was TDD nicht ist: eine Garantie für Bugfreiheit. Was es ist: eine Disziplin, die zwingt, vor dem Code über das gewünschte Verhalten nachzudenken – und die ein *Sicherheitsnetz aus Tests* entstehen lässt, das Refactoring überhaupt erst ermöglicht.

4.2 Die Testpyramide

Nicht alle Tests sind gleich. Die **Testpyramide** ordnet sie nach Geschwindigkeit, Kosten und Verlässlichkeit:

- **Unit-Tests** (Basis): viele, sehr schnell, isolieren einzelne Funktionen. Hunderte bis Tausende pro Service.
- **Integrationstests** (Mitte): weniger, prüfen Service-Grenzen und externe Abhängigkeiten (DB, andere Services). Dutzende bis hundert pro Service.
- **End-to-End-Tests** (Spitze): wenige, prüfen den Geschäftsfluss über mehrere Services. Eine Handvoll, weil teuer und fragil.

Der häufige Fehler: eine *umgekehrte Pyramide* mit vielen E2E-Tests, wenig Unit-Tests. Sie ist fragil, langsam und liefert vage Fehlermeldungen. Wer in der Pyramide investiert, investiert von unten.

4.3 Quality Gates

Ein **Quality Gate** ist eine harte Bedingung, die erfüllt sein muss, bevor ein Release in die nächste Stufe geht. Drei häufige Gates: (Forsgren et al., 2018, Kap. 3)

- **Tests grün.** Alle Tests passieren. Keine Ausnahme.
- **Static Checks.** Linting, Type-Checks, Style. Keine kosmetische Spielerei – diese Werkzeuge fangen ganze Klassen von Bugs ab.
- **Security Checks.** Abhängigkeiten ohne bekannte Sicherheitslücken, keine Secrets im Code, Container ohne kritische CVEs.

Gate-Disziplin

Ein Quality Gate, das in Krisen “mal weggelassen” wird, ist kein Gate – es ist eine Empfehlung. Gates müssen *automatisch* sein und in der Pipeline durchsetzen, nicht von Menschen prüfen. Sonst wird in Stresssituationen ihre Existenz zur Belastung statt zum Schutz.

4.4 Deployment-Pipeline

Eine moderne Deployment-Pipeline hat typischerweise diese Stationen:

1. **Commit** – Code wird gepusht, Build startet.
2. **Build** – Container-Image wird erzeugt, signiert, registriert.
3. **Unit-Tests + Static Checks** – innerhalb Minuten.
4. **Security Scan** – Abhängigkeiten, Container, Konfiguration.
5. **Integrationstests** – gegen Mock- oder Sandbox-Services.
6. **Deploy in Staging** – automatisches Hochfahren in einer produktionsähnlichen Umgebung.
7. **Smoke Tests / Canary** – minimale E2E-Prüfungen.
8. **Deploy in Produktion** – mit definierter Rollout-Strategie.

4.5 DORA-Metriken

Die **DORA-Metriken** sind heute die Standard-Messung für DevOps-Performance: (Forsgren et al., 2018, Kap. 2)

- **Lead Time for Changes** – wie lange von Commit bis Produktion?
- **Deployment Frequency** – wie oft wird in Produktion deployed?
- **Change Failure Rate** – welcher Anteil der Releases verursacht einen Incident?
- **Mean Time to Recover (MTTR)** – wie lange dauert es, bis nach einem Incident wieder Normalbetrieb herrscht?

Diese vier Metriken sind eng korreliert: leistungsfähige Teams haben *kürzere* Lead Times, *häufigere* Deployments, *niedrigere* Failure Rates und *kürzere* MTTRs. Sie ersetzen klassische Aktivitätsmessungen (“wie viele Bugs in der Story?”) durch Outcome-Messungen.

5. DevOps und Operating Model: Wer baut, wer betreibt?

DevOps ist keine Tool-Kategorie, sondern eine *Arbeitsweise*: Entwicklung und Betrieb arbeiten als ein System, nicht als zwei verfeindete Abteilungen. Das berühmte Prinzip “*You build it, you run it*” (Werner Vogels) sagt es kurz: wer baut, verantwortet auch den Betrieb. (Kim et al., 2016, Kap. 1)

5.1 Three Ways

Gene Kim formuliert die DevOps-Bewegung als “Three Ways”: (Kim et al., 2016, Einleitung)

1. **Flow.** Schneller, kontinuierlicher Wertfluss von der Idee bis zum Kunden. Engpässe sichtbar machen, Batches verkleinern, Automatisierung erhöhen.
2. **Feedback.** Schnelle, kurze Feedback-Schleifen aus dem Betrieb zurück in die Entwicklung – damit Fehler nicht zur teuren Spätereckung werden.
3. **Continuous Learning.** Eine Kultur, die Experimente erlaubt und Fehler als Lernchance nutzt – inklusive blame-freier Postmortems.

5.2 Team Topologies: vier Team-Typen

Matthew Skelton und Manuel Pais haben mit *Team Topologies* eine pragmatische Sprache für Team-Strukturen in skalierten Organisationen geprägt. Vier Team-Typen genügen: (Skelton & Pais, 2019)

- **Stream-aligned Team.** Verantwortet einen kontinuierlichen Wertstrom (z. B. einen Geschäftsprozess oder eine Produkt-Linie). Build *und* Run für ihren Bereich.
- **Platform Team.** Stellt eine interne Plattform bereit (CI/CD, Observability, Kubernetes-Standards), die Stream-aligned Teams produktiver macht.
- **Enabling Team.** Hilft anderen Teams kurzfristig, neue Fähigkeiten aufzubauen (etwa Cloud-Migration, Security-Praktiken).
- **Complicated Subsystem Team.** Spezialteam für hochkomplexe Subsysteme, die nicht jedes Stream-Team beherrschen kann (z. B. Machine-Learning-Plattform).

5.3 Operating-Model-Bausteine

Ein Operating Model beantwortet drei Fragen: *Wer macht was? Wer entscheidet was? Wie wird zusammengearbeitet?* Vier Bausteine:

1. **Produktteams (Stream-aligned).** Besitzen Services end-to-end – Build, Run, Monitor, Improve.
2. **Plattformteam.** Liefert Standards und Werkzeuge: CI/CD-Templates, Observability-Baseline, Kubernetes-Basis, Security-Defaults.
3. **Security/Compliance.** Definiert Baselines und prüft Ausnahmen. Nicht für jeden Schritt einzeln, sondern über Policies.
4. **Architektur-Governance.** Ein leichtgewichtiges Gremium, das nur teure und irreversible Entscheidungen trifft (Servicegrenzen, Datenhoheit, Plattform-Wahl).

5.4 RACI für Schnittstellenänderungen

Eine RACI-Matrix für Schnittstellenänderungen in einer Microservice-Plattform sieht oft so aus:

	Stream-Team	Plattform	Security	Architektur
Service-interne Änderung	R/A	I	C/I	I
API-Versions-Bump (additiv)	R/A	I	I	I
Breaking Change einer API	R	C	C	A
Neuer Service	R	C	C	A
Servicegrenzen-Verschiebung	C	C	I	R/A

Drei Regeln dieser Matrix: lokale Entscheidungen bleiben lokal; geteilte Verträge brauchen Konsens; teure Entscheidungen brauchen das Board. (vgl. PMI, 2021)

6. Reliability: SLOs, Error Budgets und Risiko-Architektur

Verlässlichkeit ist keine binäre Eigenschaft ("läuft / läuft nicht"), sondern ein numerisches Ziel. Das Site-Reliability-Engineering-Vokabular ist heute Standard für die Messung. (Forsgren et al., 2018, Kap. 5)

6.1 SLI, SLO, SLA

Drei Begriffe, die oft verwechselt werden:

- **SLI (Service Level Indicator)** – die konkrete Messgröße. Beispiel: "Anteil HTTP-Requests mit Statuscode 2xx und Latenz < 300 ms".
- **SLO (Service Level Objective)** – das Ziel für den SLI. Beispiel: "99,9 % über 30 Tage".
- **SLA (Service Level Agreement)** – die vertragliche Zusage, oft mit Strafzahlungen. Beispiel: "Bei weniger als 99,5 % gibt's eine Rückerstattung".

Faustregel: $SLA \leq SLO \leq SLI\text{-Realität}$. Der SLO ist enger als der SLA (Puffer-spielraum) und realistisch durch die SLI-Messung abgedeckt.

6.2 Error Budget

Ein **Error Budget** ist das Komplement zum SLO. Bei 99,9 % Verfügbarkeit darf der Service in 30 Tagen für etwa 43 Minuten ausfallen – das ist das Budget. Solange das Budget nicht verbraucht ist, darf das Team Risiken eingehen (neue

Features, Experimente). Wenn das Budget verbraucht ist, wird priorisiert auf Stabilisierung umgeschaltet.

Das Schöne an Error Budgets: sie eliminieren die endlose Diskussion “Speed vs. Safety”. Es gibt eine Regel – und die Regel entscheidet.

6.3 Change Failure Rate und MTTR

Diese zwei DORA-Metriken (siehe oben) sind die wichtigsten Risiko-Indikatoren:

- **Change Failure Rate.** Anteil der Releases, die einen Incident verursachen. Niedrige Failure Rate (z. B. unter 15 %) ist ein Indikator für eine reife Pipeline. Hohe Rate (über 30 %) deutet auf zu wenig Tests, schwache Gates oder fehlende Canary-Strategien.
- **MTTR.** Wie lange dauert es, einen Incident zu beheben? Eine niedrige MTTR (Minuten bis wenige Stunden) ist oft wichtiger als eine niedrige Fehlerrate – weil sie zeigt, dass die Organisation Probleme schnell behandelt.

Reliability als Architektur

Reliability entsteht nicht aus *Vorsicht*, sondern aus *Architektur*: gute Servicegrenzen, automatisierte Tests, klare SLOs, schnelle Rollbacks, geübte Incident-Prozesse. Wer Reliability als “mehr aufpassen” versteht, baut Bürokratie. Wer sie als “schnellere Erholung” versteht, baut Resilienz.

7. Fallstudie: InsureFast vom Monolithen zur Plattform

InsureFast ist eine mittelgroße Versicherung. Der digitale Abschluss und die Schadenmeldung laufen heute auf einem zehn Jahre alten Monolithen. Release-Zyklus: 6–8 Wochen. Probleme: häufige Produktionsausfälle, der Prozess “Schaden melden” bricht bei Lastspitzen ein, das Audit fordert Nachvollziehbarkeit (wer hat wann was entschieden). Restriktionen: 12 Wochen bis zum Saison-Peak, Budget 220 k EUR, Legacy-Datenbank, Security-Anforderungen, unklare Lastprofile.

7.1 Service-Schnitt entlang der Prozesskette

Die Prozesskette “Schaden melden → prüfen → entscheiden → auszahlen” zerfällt in sieben Service-Kandidaten:

1. **Claim Intake** – Annahme der Schadenmeldung (Web, App, Hotline). Hohe Lastvarianz.
2. **Policy / Customer** – Stamm- und Vertragsdaten. Liest bei vielen Schritten.
3. **Fraud Check** – Betrugsfrüherkennung. Externe Modelle, hohe Compute-Last.
4. **Claim Assessment** – fachliche Schadenbewertung. Menschen + Tools.
5. **Payout** – Auszahlung. Audit-kritisch, irreversibel.
6. **Notifications** – Mail, Push, SMS. Hohe Frequenz, niedriges Risiko.
7. **Document Management** – Anhänge, Korrespondenz, Archiv.

7.2 Kubernetes-Betriebskonzept

- **Rollout.** Für Notifications und Document Management: Rolling Update. Für Payout und Claim Assessment: Canary mit 5 % / 25 % / 100 %.
- **Skalierung.** CPU-basiert für die meisten Services. Claim Intake zusätzlich queue-basiert (wartet die Eingangs-Queue?).
- **Ressourcen.** Limits gesetzt, Requests aus Last-Profilen abgeleitet. Reserve für Saison-Peak vordefiniert (Cluster-Autoscaler).
- **Observability.** Zentrale Logs, Metriken pro Service, verteiltes Tracing über Claim-IDs.
- **Rollback.** Pflichtkriterium: innerhalb 10 Minuten möglich, automatisiert getestet.

7.3 Teststrategie und Quality Gates

- **Unit-Tests.** TDD für Kernlogik in Fraud Check und Claim Assessment.
- **Integrationstests.** Für jede Service-API gegen Mock-Konsumenten.
- **End-to-End.** Nur zwei kritische Pfade: “Happy Path” (Schaden gemeldet → ausgezahlt) und “Fraud Flag” (Schaden gemeldet → angehalten zur Prüfung).
- **Gates.** (1) Alle Tests grün, (2) Security/Dependency-Check ok, plus Smoke Test nach Deployment.

7.4 Operating Model

- Produktteams besitzen Services (Build + Run): Team “Claim” für Intake/Assessment, Team “Risk” für Fraud Check, Team “Pay” für Payout, Team “Communication” für Notifications/Documents.
- Plattformteam liefert CI/CD-Templates, Observability-Standard und Kubernetes-Baseline.
- Security/Compliance definiert Baselines (Datenschutz, Audit Trail, Verschlüsselung) und prüft Ausnahmen.
- Architekturboard tagt monatlich – nur für teure Entscheidungen (Servicegrenzen, Datenhoheit, Plattform-Wechsel).

7.5 Entscheidungen unter Unsicherheit

Zwei kritische Entscheidungen werden bewusst getroffen:

1. **Datenstrategie.** Statt Big-Bang-Migration bleibt die Legacy-DB zunächst bestehen. Ein **Anti-Corruption Layer** kapselt sie. Frühwarnsignal: Wenn Integrationsfehler oder Lead Time steigen, wird priorisiert auf Eigenschemata umgestellt.
2. **Release-Frequenz.** Wöchentliche Releases für nicht-kritische Services (Notifications, Documents), zweiwöchentlich für Kern (Payout, Claim Assessment). Frühwarnsignal: Steigt die Change Failure Rate über 15 %, wird die Frequenz reduziert und an der Pipeline gearbeitet.

8. Coaching: Von “Technik” zu Verantwortungsfähigkeit

Der Sprung vom Tag 1 zum Tag 10 ist nicht primär ein technischer, sondern ein Rollenwechsel: vom Modellierer zum Architekten, vom Architekten zum verantwortlichen Entscheider. Drei Coaching-Dimensionen helfen, diesen Sprung bewusst zu machen.

8.1 Welche Entscheidung ist teuer zu ändern?

In jedem Projekt gibt es Entscheidungen mit unterschiedlicher Reversibilität. Bezos hat das in seinem “Two-Way / One-Way Door”-Bild populär gemacht: *Two-Way-Door*-Entscheidungen sind reversibel, *One-Way-Door*-Entscheidungen sind teuer oder unmöglich rückgängig zu machen.

In einer Plattform-Architektur sind klassische One-Way-Door-Entscheidungen: Servicegrenzen (schwer rückgängig), Datenhoheit (sehr schwer rückgängig), Deployment-Standard (mittel-schwer), Plattform-Wahl (sehr schwer). Diese Entscheidungen verdienen ein dediziertes Architekturboard. Alles andere sollte lokal entscheidbar sein.

8.2 Wo braucht es Leitplanken, wo Autonomie?

Die Polarität “Standardisierung vs. Autonomie” ist kein Gegensatz, sondern eine Justierung. Eine bewährte Heuristik:

- **Standardisiert** (Plattform liefert): CI/CD-Pipeline-Schema, Observability-Format, Sicherheits-Baselines, Identity, Logging-Schema.
- **Autonom** (Team entscheidet): Datenbankwahl innerhalb genehmigter Optionen, Sprache/Framework innerhalb genehmigter Optionen, Service-internes Design, Test-Tools, Code-Stil.
- **Mit Board** (Architektur entscheidet): Servicegrenzen, Datenhoheit, neue Plattform-Optionen, Breaking Changes geteilter Verträge.

8.3 Risiko transparent statt versteckt

Senior-Architekten zeichnen sich nicht durch *weniger* Risiko aus, sondern durch *transparenteres* Risiko. Das heißt konkret:

- Annahmen werden dokumentiert – nicht als Schwäche-Eingeständnis, sondern als Steuerungsinformation.
- Frühwarnsignale werden vor dem Risiko-Eintritt definiert, nicht erst nach dem Vorfall.
- Eskalations-Pfade sind benannt, bevor die Eskalation nötig wird.
- “Wir wissen es nicht” wird als legitime Antwort akzeptiert – mit klarem “und so finden wir es heraus”.

Eselsbrücke: R-S-E

Drei Disziplinen der Senior-Architektur:

Reversibilität – ist diese Entscheidung One-Way oder Two-Way?

Standard vs. Autonomie – gehört das in die Plattform oder ins Team?

Explizit – ist das Risiko benannt, oder versteckt es sich in Annahmen?

Drei Fragen vor jeder Architekturentscheidung.

9. Management-Perspektive: Plattform als Entscheidungsarchitektur

Auf Wissens-Ebene ist eine Plattform eine Sammlung technischer Komponenten. Auf Anwendungs-Ebene ist sie ein laufendes System. Auf Management-Ebene ist sie eine *Entscheidungsarchitektur*: Wer darf was entscheiden? Welche Standards sind verbindlich? Welche Risiken werden bewusst eingegangen?

9.1 Top-10-Entscheidungen einer Plattform

1. **Servicegrenzen.** Welche Capabilities erhalten eigene Services?
2. **Datenhoheit.** Welcher Service besitzt welche Daten? Wer darf lesen, wer schreiben?
3. **API- und Event-Standards.** Welche Schnittstellenformate sind verbindlich (REST, gRPC, AsyncAPI)?
4. **SLO-Klassen.** Welche Services brauchen 99,9 %, welche 99,99 %, welche reichen mit 99,5 %?
5. **Release-Policy.** Wer darf wie oft deployen? Gibt es Freeze-Fenster?
6. **Security-Baselines.** Verschlüsselung, Identity, Logging, Vulnerability-Management.
7. **Observability-Standard.** Welche Logs, Metriken, Traces sind Pflicht?
8. **Incident-Governance.** Wer ist On-Call? Wie wird eskaliert? Wer schreibt das Postmortem?
9. **Ausnahmeprozess.** Wer darf vom Standard abweichen? Mit welcher Begründung? Mit welchem Review?
10. **Toolchain-Standardisierung.** CI/CD, Image-Registry, IaC-Werkzeug, Konfigurations-Management.

9.2 Zwei zentrale Zielkonflikte

1. **Speed vs. Safety.** Wöchentlich deployen heißt mehr Risiko – oder mehr Disziplin? Beides. Die Antwort liegt in *Error Budgets*: solange das Budget reicht, gilt Speed; wenn es leer ist, gilt Safety. Diese Regel ersetzt endlose Verhandlungen.
2. **Autonomie vs. Konsistenz.** Wenn jedes Team alles selbst entscheidet, wird die Plattform inkohärent. Wenn alles zentral entschieden wird, gibt es keine Autonomie mehr. Antwort: starke Standards an wenigen Stellen (Pflicht), schwache Standards (Defaults) an vielen Stellen.

9.3 Architekturprinzipien (maximal acht)

Eine Plattform braucht einen kleinen Satz Architekturprinzipien – nicht zwanzig. Beispielhaft:

1. **Capability over Layer.** Services entlang Capabilities, nicht entlang Schichten.
2. **Owner per Service.** Jeder Service hat genau einen verantwortlichen Owner.
3. **API als Vertrag.** Schnittstellen sind versioniert, Breaking Changes brauchen Übergangszeit.
4. **Observability ist Pflicht.** Kein Service ohne Logs, Metriken, Traces.

5. **Rollback in 10 Minuten.** Jedes Release ist ohne Datenmigration zurücknehmbar.
6. **Security by Default.** Verschlüsselt, authentifiziert, getrennt – ohne dass Teams es einfordern müssen.
7. **Idempotenz.** Nachrichten dürfen mehrfach zugestellt werden, ohne Schaden zu erzeugen.
8. **Document the Why.** Architekturentscheidungen werden begründet, nicht nur protokolliert (z. B. als ADR – Architecture Decision Record).

9.4 Risiken bei Plattform-Initiativen

- **Microservices-Spaghetti.** Zu viele Services, zu viele Abhängigkeiten, niemand versteht das Ganze.
- **Goldenes Plattformteam.** Die Plattform wird zum Selbstzweck, Stream-Teams nutzen sie nicht.
- **Audit-Lücke.** Die alten Compliance-Pfade greifen nicht mehr, die neuen sind nicht definiert.
- **Reife-Spreizung.** Manche Teams sind weit, andere kaum gestartet – die Plattform droht zu zerfasern.
- **Tool-Falle.** Diskussionen drehen sich um das Werkzeug statt um das Operating Model.

Schluss-Merksatz für Tag 10

Eine skalierbare digitale Prozessarchitektur ist eine **Verantwortungsarchitektur** in technischer Verpackung. Kubernetes, TDD und Microservices sind Werkzeuge. Wirksam werden sie nur durch klare Ownership, leichtgewichtige Governance und ein Operating Model, das im Incident-Fall ohne Diskussion funktioniert.

10. Take-aways aus Tag 10

1. **Servicegrenzen entlang Capabilities, nicht Schichten.** Sonst sind Microservices eine teurere Form desselben Problems.
2. **Skalierung hat drei Dimensionen:** Last, Teams, Entscheidungen. Wer eine vergisst, baut Engpässe.
3. **Kubernetes löst Orchestrierung, nicht Organisation.** Ein verteiltes System verlangt Observability, Rollback-Disziplin und Ressourcen-Limits.
4. **TDD und Testpyramide bauen Sicherheit ein.** Viele Unit-Tests, weniger Integration, wenige End-to-End.
5. **DORA-Metriken statt Aktivitätsmessung.** Lead Time, Deployment Frequency, Change Failure Rate, MTTR.
6. **Operating Model in vier Bausteinen:** Produktteams, Plattform, Security, Architektur-Governance.
7. **SLOs und Error Budgets entscheiden Speed vs. Safety** – statt endlose Verhandlungen.

8. **Architekturprinzipien (max. 8)** statt zwanzig Seiten Policy. Klar, durchsetzbar, begründet.

11. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

Beck, K. (2002).

Test driven development: By example. Addison-Wesley.

Burns, B., Villalba, E., Strebel, D., & Evenson, L. (2022).

Kubernetes: Up and running – Dive into the future of infrastructure (3rd ed.). O'Reilly.

Evans, E. (2003).

Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.

Forsgren, N., Humble, J., & Kim, G. (2018).

Accelerate: The science of lean software and DevOps – Building and scaling high performing technology organizations. IT Revolution Press.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016).

The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations. IT Revolution Press.

Newman, S. (2021).

Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly.

Project Management Institute. (2021).

A guide to the Project Management Body of Knowledge (PMBOK® Guide) (7th ed.). Project Management Institute.

Richardson, C. (2018).

Microservices patterns: With examples in Java. Manning.

Skelton, M., & Pais, M. (2019).

Team topologies: Organizing business and technology teams for fast flow. IT Revolution Press.

12. Glossar

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

Anti-Corruption Layer

Eine Kapselungsschicht zwischen neuem Service und Legacy-System, die das alte Datenmodell vor dem neuen abschirmt. Verlangsamt Big-Bang-Risiken.

API

Application Programming Interface. Vertraglich definierte Schnittstelle eines Services. Versionierung und tolerantes Lesen vermeiden Brüche.

Blue/Green Deployment

Rollout-Strategie mit zwei parallelen Umgebungen; Umschalten per Routing. Schnelles Rollback, höhere Infrastruktur-Kosten.

Bounded Context

Sprachlich und modellmäßig abgegrenzte Zone in DDD. Servicegrenzen verlaufen an den Grenzen des Bounded Context. (Evans, 2003)

Canary Release

Rollout-Strategie, bei der zuerst ein kleiner Prozentsatz der Nutzer die neue Version bekommt. Erlaubt differenziertes Risiko-Management.

Capability

Geschäftsfähigkeit; eine eigenständige Leistung der Organisation, mit klarem Input, Output und Owner.

Change Failure Rate

DORA-Metrik: Anteil der Releases, die einen Incident verursachen. Wichtiger Risiko-Indikator.

Container

Reproduzierbares Packaging-Format: Anwendung plus Abhängigkeiten in einem Artefakt.

DevOps

Arbeitsweise, die Entwicklung und Betrieb als ein System versteht ("You build it, you run it"). (Kim et al., 2016)

DORA-Metriken

Vier Kennzahlen für DevOps-Performance: Lead Time, Deployment Frequency, Change Failure Rate, MTTR. (Forsgren et al., 2018)

Error Budget

Komplement zum SLO; erlaubte Ausfallzeit pro Periode. Steuerungsmechanismus zwischen Speed und Safety.

Idempotenz

Eigenschaft eines Aufrufs: mehrfache Ausführung erzeugt dasselbe Ergebnis wie eine einfache Ausführung. Pflicht in verteilten Systemen.

Kubernetes

Container-Orchestrierungs-Plattform: Deploy, Skalierung, Self-Healing, Rollback, Service-Discovery.

Microservice

Eigenständig deploybarer Service entlang einer Capability. Eigene Daten, eigene Owner.

MTTR

Mean Time to Recover. Mittlere Zeit von Incident-Eintritt bis Normalbetrieb. Misst Resilienz.

Observability

Fähigkeit, den Zustand eines Systems von außen zu erkennen. Drei Säulen: Logs, Metriken, Traces.

Operating Model

Wer macht was, wer entscheidet was, wie wird zusammengearbeitet? Verbindet Architektur, Organisation und Governance.

Plattformteam

Internes Team, das eine Plattform für Stream-aligned Teams bereitstellt (CI/CD, Observability, Kubernetes-Basis). (Skelton & Pais, 2019)

Quality Gate

Harte Bedingung in der Pipeline (z. B. Tests grün, Security ok), die erfüllt sein muss, bevor das Release weitergeht.

Rollback

Rücknahme eines Releases auf eine vorherige Version. Pflichtkriterium: schnell und automatisierbar.

Service Discovery

Mechanismus, mit dem Services einander über logische Namen statt IP-Adressen finden.

SLO

Service Level Objective. Ziel für einen SLI, z. B. 99,9 % Verfügbarkeit über 30 Tage.

Stream-aligned Team

Team, das einen Wertstrom end-to-end verantwortet (Build und Run). (Skelton & Pais, 2019)

TDD

Test-Driven Development. Zyklus Red-Green-Refactor: erst Test, dann Code, dann Verbesserung. (Beck, 2002)

Testpyramide

Strukturprinzip: viele Unit-Tests, weniger Integration, wenige End-to-End. Schnell, robust, kostengünstig.